Лекция 2. Транспортные протоколы.

1. Место транспортных протоколов в сетевой модели

Любое взаимодействие между программными системами — будь то вызов API, передача файла, потоковая передача видео или обмен сообщениями между микросервисами — опирается на транспортные протоколы. Они занимают четвёртый уровень модели OSI и обеспечивают передачу данных между приложениями, работающими на разных узлах сети.

Транспортный уровень решает фундаментальные задачи: как доставить данные от отправителя к получателю, как обеспечить надёжность, как управлять потоком и перегрузкой, как мультиплексировать несколько соединений. От того, какой протокол выбран, напрямую зависят задержка, пропускная способность, устойчивость к потерям и безопасность приложения.

В современной разработке выбор транспортного протокола — это не просто техническая деталь. Это архитектурное решение, которое определяет, сможет ли система масштабироваться, выдержит ли она нагрузку, будет ли она работать в условиях нестабильной сети. Ошибка в выборе протокола может привести к тому, что приложение будет работать медленно, потреблять избыточные ресурсы или вообще не справляться с задачей.

В этой лекции мы разберём основные транспортные протоколы, их характеристики, области применения и критерии выбора.

Транспортный протокол — это набор правил и форматов, определяющих способ передачи данных между приложениями через сеть. Протокол описывает, как устанавливается соединение, как передаются данные, как подтверждается получение, как обрабатываются ошибки.

Транспортная служба (Transport Service) — это набор транспортных функций, предоставляемых приложению, без привязки к конкретному формату протокола. Один и тот же набор служб может быть реализован разными протоколами.

Соединение (Connection) — логическая связь между двумя узлами, в рамках которой передаются данные. Соединение может быть установлено явно (как в TCP) или отсутствовать (как в UDP).

Надёжность доставки — способность протокола гарантировать, что данные дойдут до получателя в правильном порядке и без потерь. Надёжность может быть полной (гарантированная доставка), частичной (доставка с повторами) или отсутствовать (best-effort).

Мультиплексирование — возможность передавать данные нескольких логических потоков через одно соединение.

Управление потоком (Flow Control) — механизм, предотвращающий переполнение буфера получателя.

Управление перегрузкой (Congestion Control) — механизм, регулирующий скорость отправки данных в зависимости от состояния сети.

2. **TCP: протокол с установлением соединения**

2.1. Общая характеристика

TCP (Transmission Control Protocol) — это стандартный транспортный протокол IETF, описанный в RFC 793. С момента своего появления TCP стал основным протоколом с установлением соединения и потоковой передачей данных в Интернете. Он широко реализован в конечных системах и используется большинством приложений.

2.2. Принципы работы

TCP — это connection-oriented протокол. Перед началом передачи данных выполняется трёхстороннее рукопожатие (three-way handshake), позволяющее клиенту и серверу установить соединение и согласовать параметры. Также предусмотрены механизмы для упорядоченного завершения и немедленного разрыва соединения.

TCP обеспечивает мультиплексирование через номера портов. Активная TCP-сессия идентифицируется четырёхкортежем: локальный IP-адрес, удалённый IP-адрес, локальный порт, удалённый порт. Порт назначения при установке соединения часто используется для указания запрашиваемой службы.

TCP разбивает непрерывный поток байтов на сегменты, размер которых соответствует IP-пакетам. Каждый байт в потоке идентифицируется порядковым номером. Порядковый номер используется для упорядочивания сегментов при получении, идентификации сегментов в подтверждениях и обнаружения неподтверждённых сегментов для повторной передачи. Это основа надёжной, упорядоченной доставки данных.

Выборочное подтверждение (SACK) расширяет этот механизм, позволяя раньше идентифицировать, какие сегменты потеряны, что ускоряет повторную передачу и снижает количество избыточных повторов.

Управление потоком в TCP реализуется через скользящее окно: ограничивается объём неподтверждённых данных, которые могут быть отправлены в данный момент. Опция масштабирования окна позволяет использовать окна размером более 64 КБ.

Управление перегрузкой — обязательная функция всех TCP-отправителей. Отправитель использует порядковый номер и скользящее окно приёмника для управления отдельным окном перегрузки. Отправляющее окно в каждый момент времени — это минимум из окна приёмника и окна перегрузки. Окно перегрузки увеличивается при отсутствии перегрузки и уменьшается при её обнаружении.

Обнаружение перегрузки в TCP часто основано на потерях. Используются два механизма: таймер повторной передачи с экспоненциальной задержкой и получение трёх дублированных подтверждений (fast retransmit). Также поддерживается Explicit Congestion Notification (ECN), позволяющий сетевому узлу сигнализировать о перегрузке без потери пакетов.

2.3. Ограничения TCP

Несмотря на надёжность и универсальность, TCP имеет ограничения. Главное из них — блокировка головного пакета (Head-of-Line Blocking). Если один сегмент потерян, все последующие данные не могут быть доставлены приложению, пока потерянный сегмент не будет повторно передан и получен. Это критично для приложений, требующих низкой задержки.

Кроме того, TCP требует нескольких round-trip time (RTT) для установления соединения и не имеет встроенной безопасности на транспортном уровне.

3. UDP: протокол без установления соединения

3.1. Общая характеристика

UDP (User Datagram Protocol) — это простой транспортный протокол, описанный в RFC 768. Он создан Д. П. Ридом в 1980 году**. UDP отправляет сообщения через IP-сеть с использованием датаграмм.** Это протокол без установления соединения, поэтому предварительная коммуникация для установки каналов или путей не требуется.

3.2. Принципы работы

UDP — lightweight протокол: у него нет рукопожатия, что делает его подходящим для ресурсоограниченных IoT-устройств. Однако у него есть существенные недостатки: ненадёжность, отсутствие гарантии доставки, отсутствие упорядочивания и защиты от дублированных сообщений. UDP обеспечивает целостность данных с помощью контрольных сумм и использует номера портов для адресации функций на источнике и приёмнике.

UDP не предоставляет поддержку сегментации на уровне протокола. При передаче UDP инкапсулирует каждую датаграмму в один IP-пакет или несколько фрагментов IP-пакета. Это позволяет датаграмме быть больше эффективного MTU пути.

3.3. Области применения

UDP используется там, где важнее скорость и минимальная задержка, чем гарантированная доставка: потоковое видео, онлайн-игры, VoIP, DNS-запросы, телеметрия. Если приложению нужна надёжность, оно должно реализовать её самостоятельно поверх UDP.

4. **QUIC: современный транспортный протокол**

4.1. Общая характеристика

QUIC (Quick UDP Internet Connections) — это инновационный транспортный протокол общего назначения, первоначально разработанный Джимом Роскиндом в Google и развёрнутый в 2012 году. Публично анонсирован IETF в 2013 году, стандартизован в 2021 году (RFC 9000). QUIC работает поверх UDP, унаследовав от него малые задержки и гибкость, но дополняет его механизмами, характерными для TCP.

4.2. **Ключевые особенности**

Минимизация времени установления соединения. Первая связь клиент–сервер устанавливается за один обмен пакетами (1-RTT). При повторных соединениях QUIC поддерживает режим 0-RTT, позволяющий отправлять данные сразу без полного рукопожатия, используя ранее согласованные ключи шифрования.

Устранение Head-of-Line Blocking. QUIC поддерживает мультиплексирование потоков внутри одного соединения. Потеря отдельного пакета не останавливает передачу других потоков.

Встроенная безопасность. QUIC всегда шифрует трафик: интеграция с TLS 1.3 осуществляется на уровне самого протокола. Метаданные сессии скрыты от промежуточных узлов, что повышает конфиденциальность и безопасность передачи.

Мобильность соединения. QUIC вводит понятие идентификаторов соединения, что позволяет соединению сохраняться при изменении IP-адреса или переходе устройства между сетями (например, с Wi-Fi на мобильную сеть). Такая мобильность недостижима для TCP-соединений.

Гибкость управления перегрузкой. Встроенные алгоритмы контроля перегрузки и потока в QUIC настраиваемы и способны эволюционировать без риска «окостенения». Протокол шифрует служебные заголовки, избегая жёсткой привязки сетевого оборудования к конкретным реализациям.

4.3. Применение

Именно QUIC выбран в качестве основы нового стандарта HTTP/3, применяемого в браузерах Chrome, Firefox, Safari. Крупные платформы, такие как YouTube и Google Search, передают контент с использованием QUIC, добиваясь ускорения загрузки на 20–30% и более. По данным Meta, около 75% трафика компании идёт через QUIC/HTTP3. В целом более 40% веб-трафика передаётся через QUIC/HTTP/3.

5. **Протоколы прикладного уровня и их транспортная основа**

Транспортные протоколы редко используются напрямую приложениями. Чаще всего они служат основой для протоколов более высокого уровня. Рассмотрим основные.

5.1. HTTP/HTTPS

HTTP (Hypertext Transfer Protocol) — протокол прикладного уровня, работающий поверх TCP (HTTP/1.1, HTTP/2) или QUIC (HTTP/3). Это запрос-ответный протокол с текстовым форматом. Используется для веб-API, RESTful-сервисов. **HTTPS — это HTTP поверх TLS, обеспечивающий шифрование.**

HTTP-запросы могут быть «тяжёлыми», особенно в текстовом формате. Эффективность зависит от оптимизаций и кеширования. HTTP легко расширяется с использованием RESTful API. Часто используется для микросервисной архитектуры при взаимодействии с внешними клиентами.

5.2. WebSocket

WebSocket — протокол, обеспечивающий постоянное соединение для двусторонней коммуникации. Работает поверх TCP. Используется для приложений реального времени: чаты, биржевые котировки, онлайн-игры. Сообщения могут быть текстовыми или бинарными. WebSocket эффективен для реального времени и легко масштабируется.

5.3. gRPC

gRPC — высокопроизводительный RPC-фреймворк от Google. Использует HTTP/2 в качестве транспорта и Protocol Buffers (protobuf) для сериализации данных. Это бинарный протокол, обеспечивающий компактную передачу данных и эффективную сериализацию/десериализацию. gRPC поддерживает потоковую передачу (в том числе двунаправленную), имеет отличную мультиязычную поддержку. Браузеры не поддерживают gRPC напрямую, требуется gRPC-Web.

gRPC рекомендуется для микросервисной архитектуры благодаря бинарному формату и эффективности. Используется для внутренних вызовов между сервисами, высоконагруженных сценариев, реального времени.

5.4. MQTT

MQTT — лёгкий протокол публикации/подписки, оптимизированный для IoT и устройств с ограниченными ресурсами. Работает поверх TCP. Сообщения лёгкие, эффективность высокая для большого количества устройств. MQTT не хранит сообщения: если нет подписчиков, сообщение отбрасывается. Используется для телеметрии, сенсорных сетей, edge-вычислений.

5.5. AMQP

AMQP (Advanced Message Queuing Protocol) — протокол для надёжного обмена сообщениями между приложениями. Поддерживает очереди, маршрутизацию, публикацию/подписку, запрос-ответ. Обеспечивает гарантии доставки (at-most-once, at-least-once). Реализации: RabbitMQ, Azure Service Bus, IBM MQ. Используется для корпоративной интеграции, межъязыкового взаимодействия.

5.6. Kafka

Apache Kafka — не просто протокол, а распределённая платформа для потоковой передачи данных. Использует собственный бинарный протокол. Ключевая особенность — распределённый append-only лог с возможностью хранения и повторного воспроизведения сообщений. Kafka обеспечивает очень высокую пропускную способность и используется для потоковой обработки, event sourcing, аналитики в реальном времени.

6. Критерии выбора транспортного протокола

Выбор протокола определяется требованиями приложения. Рассмотрим основные критерии.

Надёжность доставки. Если данные критичны и потеря недопустима — выбирайте TCP или QUIC. Если потеря допустима (видео, игры) — UDP.

Задержка. Для приложений реального времени (менее 50 мс) QUIC предпочтительнее TCP благодаря 1-RTT/0-RTT и устранению Head-of-Line Blocking. UDP обеспечивает минимальную задержку, но требует самостоятельной реализации надёжности.

Пропускная способность. Для высоких объёмов данных выбирайте QUIC или TCP с оптимизированными алгоритмами управления перегрузкой (BBRv2). Для потоковой обработки — Kafka.

Безопасность. QUIC имеет встроенное шифрование. TCP требует внешнего TLS. UDP не имеет безопасности на транспортном уровне.

Мобильность. QUIC сохраняет соединение при смене сети. TCP этого не поддерживает.

Ресурсы устройства. Для IoT и ограниченных устройств MQTT или UDP предпочтительнее, чем TCP или QUIC.

Совместимость. HTTP/HTTPS универсальны и поддерживаются всеми браузерами. gRPC требует gRPC-Web для браузеров. MQTT требует брокера.

7. Практические аспекты

7.1. **Типичные ошибки при выборе**

Использование TCP там, где нужна низкая задержка. TCP может добавлять задержки из-за Head-of-Line Blocking и рукопожатий. Для实时-приложений это критично.

Использование UDP без реализации надёжности. UDP не гарантирует доставку. Если приложение ожидает надёжности, её нужно реализовать самостоятельно или использовать QUIC.

Игнорирование ограничений сети. В некоторых сетях HTTP/2 может блокироваться. gRPC может не работать через прокси.

Выбор «популярного» протокола. Не стоит выбирать gRPC только потому, что он модный, если приложение простое и RESTful вполне достаточно.

7.2. Лучшие практики

Внешние API — HTTP/REST. Для публичных API, браузерных клиентов и простой интеграции HTTP/HTTPS остаётся стандартом.

Внутренние сервисы — gRPC. Для микросервисов, где важны производительность и низкая задержка, gRPC предпочтительнее.

Асинхронная коммуникация — MQ. Для событий и асинхронной обработки используйте брокеры сообщений (AMQP, Kafka, MQTT).

Реальное время — WebSocket или QUIC. Для чатов, стриминга, игр.

IoT — MQTT. Для устройств с ограниченными ресурсами.

Современный веб — QUIC/HTTP/3. Для ускорения загрузки и снижения задержек.

8. Домашнее задание

1. Выберите любое веб-приложение, которым вы пользуетесь ежедневно (социальная сеть, мессенджер, видеосервис). Определите, какие транспортные и прикладные протоколы оно использует.

2. Обоснуйте, почему для этого приложения выбран именно такой стек. Какие альтернативы могли бы быть использованы и почему они не подходят?

3. Опишите, какие преимущества дало бы переход на QUIC/HTTP/3 для этого приложения. Какие ограничения могли бы возникнуть?

4. Приведите пример приложения, для которого UDP подходит лучше, чем TCP. Обоснуйте выбор.